arXiv:2602.16284v1 [cs.LG] 18 Feb 2026

Fast KV Compaction via Attention Matching

Adam Zweiger [1] **Xinghong Fu** [1] **Han Guo** [1] **Yoon Kim** [1]

https://github.com/adamzweiger/compaction

Abstract

Scaling language models to long contexts is often bottlenecked by the size of the key–value (KV) cache. In deployed settings, long contexts are typically managed through *compaction* in token space via summarization. However, summarization can be highly lossy, substantially harming downstream performance. Recent work on Cartridges (Eyuboglu et al., 2025) has shown that it is possible to *train* highly compact KV caches in latent space that closely match full-context performance, but at the cost of slow and expensive end-to-end optimization. This work describes an approach for *fast* context compaction in latent space through **Attention Matching**, which constructs compact keys and values to reproduce attention outputs and preserve attention mass at a per-KV-head level. We show that this formulation naturally decomposes into simple subproblems, some of which admit efficient closed-form solutions. Within this framework, we develop a family of methods that significantly push the Pareto frontier of compaction time versus quality, achieving up to $50\times$ compaction in seconds on some datasets with little quality loss.

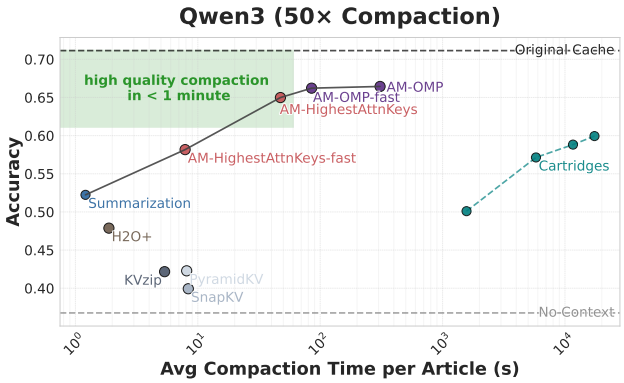

Figure 1. **Accuracy vs. Compaction Time Trade-off (Qwen3-4B; QuALITY).** We compare downstream QA accuracy ($n = 894$) after compaction, plotted against the average wall-clock time required to compact a context (seconds, log-scale) using a single H100 GPU at a fixed $50\times$ compaction ratio. Our attention-matching (AM) methods trace a speed–quality tradeoff and form the Pareto frontier, outperforming prior token-selection baselines and exceeding the performance of Cartridges (Eyuboglu et al., 2025) while being 2 orders of magnitude faster; additional Cartridges training may further improve its results.

1. Introduction

Memory has emerged as a critical bottleneck in modern language models (LMs). As such systems are deployed in increasingly long-horizon settings—reasoning, multi-session dialogue, long-running agentic coding—the question of how to efficiently manage, compact, and retrieve contextual information has become paramount. The field is converging on a consensus: models that can remember more, and remember better, will unlock capabilities that remain out of reach for systems constrained by fixed context windows.

For autoregressive LMs based on the Transformer architecture, the memory bottleneck is specific: the key-value (KV) cache. Models must retain keys and values from all previous tokens, and in long-context settings, the KV cache can reach many gigabytes per request. Existing approaches to KV cache reduction, such as token eviction (Zhang et al., 2023; Li et al., 2024; Kim et al., 2025b), token merging (Wang et al., 2024; Zhang et al., 2024), and head sparsification (Xiao et al., 2024; 2025), degrade rapidly at high reduction ratios. As a result, real-world systems still rely heavily on summarizing or simply dropping older context (Anthropic, 2025; OpenAI, 2025). Effective *context compaction*[1]—reducing KV cache size in a single pass while preserving downstream model behavior—remains an important open problem.

[1]Massachusetts Institute of Technology. Correspondence to: Adam Zweiger <adamz@mit.edu>.

Preprint.

[1]We use "compaction" (instead of "compression") by analogy to log or memory compaction in systems, where many records are consolidated into a smaller representation that preserves the information needed for future access. The term "compaction" is similarly used by agentic coding tools (e.g., Claude Code, OpenAI Codex) to describe this operation, historically implemented purely via summarization.

Recently, Eyuboglu et al. (2025) propose Cartridges, an approach that can be interpreted as performing context compaction in *latent space*. Cartridges uses prefix-tuning (Li & Liang, 2021) on synthetic "self-study" data to *train* a compact KV cache for a given context. This enables compaction ratios as high as $50\times$ on long contexts with minimal performance loss. However, its end-to-end gradient-based optimization strategy can be prohibitively expensive, often requiring several GPU-hours to train a single Cartridge for a given context.

This paper describes a family of methods for fast KV compaction that can achieve Cartridges-level compaction ratios and quality while being orders of magnitude faster. Our approach is based on an **Attention Matching** objective: rather than training a compact KV cache end-to-end on output likelihoods, we directly optimize for compacted keys and values to reproduce the *attention outputs* and *attention mass* for every KV-head in every layer, matching these on a set of reference queries. We show that this Attention Matching objective decomposes into simple subroutines that admit closed-form solutions that are efficient to compute in practice, allowing us to avoid gradient descent entirely at compaction time. Different design choices within our framework yield a family of methods along the speed–performance frontier. Attention Matching enables compaction that is orders of magnitude faster than gradient-based optimization (e.g., minutes rather than hours), with little performance degradation at ratios up to $50\times$ (see Figure 1).

2. KV Compaction via Attention Matching

Consider the problem of compacting T tokens from a context. Let $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{T\times d}$ denote the corresponding keys and values for a single KV-head, where d is the head dimension. Rotary embeddings are assumed to have already been applied to cached keys.

KV compaction aims to replace $(\boldsymbol{K}, \boldsymbol{V})$ with a shorter cache $(\boldsymbol{C}_k, \boldsymbol{C}_v) \in \mathbb{R}^{t\times d}$ with $t < T$ such that conditioning on $(\boldsymbol{C}_k, \boldsymbol{C}_v)$ behaves similarly to conditioning on $(\boldsymbol{K}, \boldsymbol{V})$ for any query $\boldsymbol{q} \in \mathbb{R}^{1\times d}$.

A key requirement is compatibility with ordinary KV caching: compaction should remain valid even when the compacted prefix is concatenated with arbitrary uncompacted tokens (e.g., the most recent turn) or with future tokens appended after compaction (e.g., user queries or model continuations). Concretely, for any additional key–value blocks $(\boldsymbol{K}_{\text{fixed}}, \boldsymbol{V}_{\text{fixed}})$, we would ideally like

$$\mathrm{Attn}\left(\boldsymbol{q}; \begin{bmatrix}\boldsymbol{K}\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}, \begin{bmatrix}\boldsymbol{V}\\ \boldsymbol{V}_{\text{fixed}}\end{bmatrix}\right) \approx \mathrm{Attn}\left(\boldsymbol{q}; \begin{bmatrix}\boldsymbol{C}_k\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}, \begin{bmatrix}\boldsymbol{C}_v\\ \boldsymbol{V}_{\text{fixed}}\end{bmatrix}\right).$$

Written explicitly (omitting the $\sqrt{d}$ scaling for readability; see Appendix A), we would like

$$\frac{\exp\left(\boldsymbol{q}\begin{bmatrix}\boldsymbol{K}\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}^\top\right)\begin{bmatrix}\boldsymbol{V}\\ \boldsymbol{V}_{\text{fixed}}\end{bmatrix}}{\sum_{j=1}^{T+S}\exp\left(\boldsymbol{q}\begin{bmatrix}\boldsymbol{K}\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}^\top_j\right)} \approx \frac{\exp\left(\boldsymbol{q}\begin{bmatrix}\boldsymbol{C}_k\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}^\top\right)\begin{bmatrix}\boldsymbol{C}_v\\ \boldsymbol{V}_{\text{fixed}}\end{bmatrix}}{\sum_{j=1}^{t+S}\exp\left(\boldsymbol{q}\begin{bmatrix}\boldsymbol{C}_k\\ \boldsymbol{K}_{\text{fixed}}\end{bmatrix}^\top_j\right)}.$$

Directly enforcing this for all possible future $(\boldsymbol{K}_{\text{fixed}}, \boldsymbol{V}_{\text{fixed}})$ is challenging. Instead, we use the fact that attention over concatenated blocks decomposes into a mixture of each block's *locally normalized* attention output, weighted by that block's *attention mass*.[2] For a key block $\boldsymbol{K}$, define

$$\mathrm{Mass}(\boldsymbol{q}; \boldsymbol{K}) \;=\; \sum_j \exp\left(\boldsymbol{q}\boldsymbol{K}_j^\top\right).$$

This suggests a sufficient compaction strategy that does not depend on unknown future tokens: over a set of reference (or "training") queries, match (i) the compacted block's local attention output and (ii) its attention mass.

One subtlety is that with $t < T$, exact mass matching using only $\boldsymbol{C}_k$ is impossible. For example, for $\boldsymbol{q} = \boldsymbol{0}$ we have $\mathrm{Mass}(\boldsymbol{0}; \boldsymbol{K}) = T$, whereas $\mathrm{Mass}(\boldsymbol{0}; \boldsymbol{C}_k) = t$ for any $\boldsymbol{C}_k \in \mathbb{R}^{t\times d}$. We therefore introduce a per-token scalar bias $\boldsymbol{\beta} \in \mathbb{R}^t$, which multiplicatively reweights the contribution of each retained key to the mass. We later show that these scalar biases arise naturally in Attention Matching.

Specifically, we optimize $(\boldsymbol{C}_k, \boldsymbol{\beta}, \boldsymbol{C}_v)$, where $\boldsymbol{C}_k, \boldsymbol{C}_v \in \mathbb{R}^{t\times d}$ and $\boldsymbol{\beta} \in \mathbb{R}^t$ so that for all queries $\boldsymbol{q}$ of interest,

$$\frac{\exp\left(\boldsymbol{q}\boldsymbol{K}^\top\right)\boldsymbol{V}}{\sum_{j=1}^{T}\exp\left(\boldsymbol{q}\boldsymbol{K}_j^\top\right)} \approx \frac{\exp\left(\boldsymbol{q}\boldsymbol{C}_k^\top + \boldsymbol{\beta}\right)\boldsymbol{C}_v}{\sum_{j=1}^{t}\exp\left(\boldsymbol{q}(\boldsymbol{C}_k)_j^\top + \boldsymbol{\beta}_j\right)}, \tag{1}$$

$$\sum_{j=1}^{T}\exp\left(\boldsymbol{q}\boldsymbol{K}_j^\top\right) \approx \sum_{j=1}^{t}\exp\left(\boldsymbol{q}(\boldsymbol{C}_k)_j^\top + \boldsymbol{\beta}_j\right). \tag{2}$$

Equation (1) matches the compacted block's local attention output, while Eq. (2) matches its attention mass; together, these preserve the block's contribution under concatenation with arbitrary fixed or future tokens (Appendix A.2). In contrast, methods that only drop or merge tokens without biases (e.g., evicting $T - t$ tokens) systematically underestimate the compacted block's contribution during future decoding.

The scalar biases add negligible memory overhead (an additional factor of $\frac{2d+1}{2d}$) and negligible-to-zero change in attention runtime. They are supported in common attention implementations such as PyTorch SDPA and FlexAttention (Dong et al., 2025).

[2] Efficient attention implementations such as FlashAttention (Dao et al., 2022) and Cascade Inference (Ye et al., 2024; Juravsky et al., 2024) exploit the same decomposition.

Finally, although the compacted cache stores only t KV entries, it retains a *logical length* T so that newly appended tokens receive the same position IDs (and thus the same RoPE phases) as they would under the uncompacted prefix. We view this as disentangling the cumulative length a cache has *seen* from its physical size.

3. Methods

We now present a family of methods for constructing $(\boldsymbol{C}_k, \boldsymbol{\beta}, \boldsymbol{C}_v)$. Our approach is defined with respect to a set of *reference queries* $\boldsymbol{Q}_{\text{ref}} = [\boldsymbol{q}_1; \ldots; \boldsymbol{q}_n] \in \mathbb{R}^{n \times d}$, which serve as a proxy for the queries the model is likely to produce when attending to the context.

Jointly optimizing $(\boldsymbol{C}_k, \boldsymbol{\beta}, \boldsymbol{C}_v)$ is computationally difficult and typically requires gradient-based optimization, which we aim to avoid at compaction time. Instead, we first construct the compacted keys $\boldsymbol{C}_k$, then compute the bias terms $\boldsymbol{\beta}$, and finally obtain $\boldsymbol{C}_v$, all in closed form.

3.1. Sampling Reference Queries $\boldsymbol{Q}_{\text{ref}}$

We consider two main approaches for constructing $\boldsymbol{Q}_{\text{ref}}$.

Repeat-prefill. Following KVzip (Kim et al., 2025b), we construct a sequence

"$\{\mathcal{C}\}$ Repeat the previous context. $\{\mathcal{C}\}$"

(with the model's chat template applied appropriately), run a prefill pass on this sequence, and extract the query vectors used while the model reconstructs $\mathcal{C}$ (i.e., query activations starting from the instruction through the second $\mathcal{C}$). We found that the simpler variant of running a prefill on $\mathcal{C}$ alone, which we call `context-prefill` (and used by H2O (Zhang et al., 2023)), was cheaper (see Table 1) but performed slightly worse than `repeat-prefill`.

Self-study. Self-study (Eyuboglu et al., 2025) generates synthetic interactions conditioned on a fixed context $\mathcal{C}$. We use it as a lightweight way to broaden the query distribution: we prompt the model with $\mathcal{C}$ and a small set of fixed prompts (e.g., "Aggregate all key facts mentioned in the context") or model-generated conversation-starters. We then sample responses and extract the resulting query vectors. In practice, we run only four prompts (Appendix C.4), with the majority of reference queries coming from `repeat-prefill`.

These approaches concentrate reference queries on representations the model naturally produces when processing and reasoning about $\mathcal{C}$. Empirically, in Appendix D, we observe that `self-study` yields the best downstream performance; `context-prefill` and `repeat-prefill` are nearly as good while being faster. We also find that randomly sampling $\boldsymbol{q}_i \sim \mathcal{N}(0, \boldsymbol{I}_d)$ (`random-vectors`) works, though it lags the other approaches.

"On-policy" queries. Per-layer compaction can induce distribution shift in queries: compacting early layers changes the residual stream seen by later layers, so the queries they produce may differ from those extracted from the unmodified model. To reduce this mismatch, we compact layers sequentially and, for each layer ℓ, extract $\boldsymbol{Q}_{\text{ref}}^{\ell}$ by running the model with layers $< \ell$ already compacted, then optimize compaction at layer ℓ using these on-policy queries. This yields slight but consistent improvements.

3.2. Constructing $\boldsymbol{\beta}$ and $\boldsymbol{C}_v$

Before describing how we construct compact keys $\boldsymbol{C}_k$, we first explain how we learn $\boldsymbol{\beta}$ and $\boldsymbol{C}_v$ to satisfy the attention-matching objectives (Eqs. 1–2) given reference queries $\boldsymbol{Q}_{\text{ref}} = [\boldsymbol{q}_1; \ldots; \boldsymbol{q}_n] \in \mathbb{R}^{n \times d}$ and compacted $\boldsymbol{C}_k$. (In one of the variants we consider, we alternate between fitting subsets of $\boldsymbol{C}_k$ and $\boldsymbol{\beta}, \boldsymbol{C}_v$.)

Fitting $\boldsymbol{\beta}$. Let $\mathbf{m} = [m_1, \ldots, m_n]^\top$ be the vector of original attention mass, i.e.,

$$m_i = \sum_{k=1}^{T} \exp(\boldsymbol{q}_i \boldsymbol{K}_k^\top).$$

Parameterizing $w_j = \exp(\boldsymbol{\beta}_j) > 0$, we would then like

$$m_i \approx \sum_{j=1}^{t} \exp\big(\boldsymbol{q}_i (\boldsymbol{C}_k)_j^\top + \boldsymbol{\beta}_j\big) = \sum_{j=1}^{t} w_j \exp\big(\boldsymbol{q}_i (\boldsymbol{C}_k)_j^\top\big).$$

This results in the following optimization problem

$$\min_{\mathbf{w}:\, w_j \geq 0} \|\boldsymbol{A}\mathbf{w} - \mathbf{m}\|_2^2$$

where $\boldsymbol{A}_{ij} = \exp\big(\boldsymbol{q}_i (\boldsymbol{C}_k)_j^\top\big)$ and $\mathbf{w} = [w_1, \ldots, w_t]^\top$. We solve for this via nonnegative least squares (NNLS; see Appendix C.2 for implementation details), and then set $\boldsymbol{\beta}_j = \log(w_j)$, clamping w_j to a small positive value if needed. Intuitively, w_j represents how many original keys' worth of attention mass the compact key $(\boldsymbol{C}_k)_j$ accounts for.

Fitting $\boldsymbol{C}_v$. Recall that the attention-output matching condition for a query $\boldsymbol{q}$ is given by,

$$\frac{\exp(\boldsymbol{q}\boldsymbol{K}^\top)\boldsymbol{V}}{\sum_{j=1}^{T} \exp(\boldsymbol{q}\boldsymbol{K}_j^\top)} \approx \left(\frac{\exp\big(\boldsymbol{q}\boldsymbol{C}_k^\top + \boldsymbol{\beta}\big)}{\sum_{j=1}^{t} \exp\big(\boldsymbol{q}(\boldsymbol{C}_k)_j^\top + \boldsymbol{\beta}_j\big)} \right) \boldsymbol{C}_v$$

With $\boldsymbol{C}_k$ and $\boldsymbol{\beta}$ fixed, we can solve for $\boldsymbol{C}_v$ with ordinary least squares. For each reference query $\boldsymbol{q}_i$ we compute

$$y_i = \frac{\exp(\boldsymbol{q}_i \boldsymbol{K}^\top)\boldsymbol{V}}{\sum_{j=1}^{T} \exp(\boldsymbol{q}_i \boldsymbol{K}_j^\top)} \in \mathbb{R}^{1 \times d},$$

$$x_i = \frac{\exp\big(\boldsymbol{q}_i \boldsymbol{C}_k^\top + \boldsymbol{\beta}\big)}{\sum_{j=1}^{t} \exp\big(\boldsymbol{q}_i (\boldsymbol{C}_k)_j^\top + \boldsymbol{\beta}_j\big)} \in \mathbb{R}^{1 \times t}.$$

Stacking into $\boldsymbol{Y} = [y_1; \ldots; y_n] \in \mathbb{R}^{n \times d}$ and $\boldsymbol{X} = [x_1; \ldots; x_n] \in \mathbb{R}^{n \times t}$, we solve for $\boldsymbol{C}_v$ with

$$\boldsymbol{C}_v^\star = \arg\min_{\boldsymbol{C}_v} \|\boldsymbol{X}\boldsymbol{C}_v - \boldsymbol{Y}\|_F^2, \tag{3}$$

$$= (\boldsymbol{X}^\top \boldsymbol{X})^{-1} \boldsymbol{X}^\top \boldsymbol{Y}. \tag{4}$$

In summary, given any set of compacted keys $\boldsymbol{C}_k$, simple linear-algebra routines—least squares and nonnegative least squares—allow us to learn $(\boldsymbol{\beta}, \boldsymbol{C}_v)$ that minimize ℓ_2 error in the attention mass and attention outputs over the reference queries. Now the main challenge becomes selecting $\boldsymbol{C}_k$.

3.3. Selecting $\boldsymbol{C}_k$

We do not have a closed-form solution for constructing $\boldsymbol{C}_k$ in general. However we found it empirically effective (and efficient) to restrict $\boldsymbol{C}_k$ to be a subset of the original keys, i.e., $\boldsymbol{C}_k = \boldsymbol{K}_{S,:}$ for some index set $S \subset \{1, \ldots, T\}$ with $|S| = t$. This allows us to avoid iterative gradient-based optimization. We consider two methods for choosing this subset, which have different efficiency-performance tradeoffs.

Highest attention keys. A simple approach in prior work on KV cache eviction/pruning (Zhang et al., 2023; Li et al., 2024; Kim et al., 2025b) is to retain keys that receive the most attention. In our setting, we measure this under the reference queries. For each reference query $\boldsymbol{q}_i$, we compute attention weights over the original keys:

$$a_i = \mathrm{softmax}(\boldsymbol{q}_i \boldsymbol{K}^\top) \in \mathbb{R}^{1 \times T}.$$

We then aggregate these weights across queries via root mean square over $(a_{1,j}, \ldots, a_{n,j})$ to obtain a per-key importance score s_j. We found RMS to be more robust than mean or max aggregation (Appendix F.1).

Orthogonal matching pursuit (OMP) keys. The method above is a fast and effective heuristic in practice. As a more direct alternative, we can explicitly match the attention mass using orthogonal matching pursuit (OMP; Tropp & Gilbert, 2007), which greedily builds $\boldsymbol{C}_k$ and $\boldsymbol{\beta}$ to best satisfy Eq. 2.

Define the mass feature matrix $\boldsymbol{\Phi} \in \mathbb{R}^{n \times T}$ with $\boldsymbol{\Phi}_{ij} = \exp(\boldsymbol{q}_i \boldsymbol{K}_j^\top)$ and target vector $\mathbf{m}$ with $m_i = \sum_j \boldsymbol{\Phi}_{ij}$. We seek a sparse subset S and weights $\mathbf{w} \geq 0$ minimizing $\|\boldsymbol{\Phi}_{:,S}\mathbf{w} - \mathbf{m}\|_2^2$. OMP selects S greedily: at each step, it adds the key whose column maximally reduces the residual, then refits $\mathbf{w}$ via NNLS (Algorithm 1). This algorithm directly yields $\boldsymbol{\beta} = \log \mathbf{w}$; we then fit $\boldsymbol{C}_v$ via least squares as above.

While OMP performs best empirically, it is slower than the other methods: greedy selection with NNLS refitting scales at least linearly in t. In practice, selecting multiple keys per step and refitting at intervals reduces compaction time by 4–8× with little degradation (Appendix C.1).

Algorithm 1 OMP Key Selection

Require: Original keys $\boldsymbol{K} \in \mathbb{R}^{T \times d}$, queries $\boldsymbol{Q} \in \mathbb{R}^{n \times d}$, budget t
Ensure: Indices S, weights $\mathbf{w}$ (with $\boldsymbol{\beta} = \log \mathbf{w}$)

```
Φ_ij ← exp(q_i K_j^⊤ / √d) {Mass feature matrix}
m_i ← Σ_{j=1}^{T} Φ_ij {Target mass vector}
r ← m,   S ← ∅
for k = 1 to t do
    j* ← arg max_{j∉S} (r^⊤ Φ_{:,j})
    S ← S ∪ {j*}
    w ← arg min_{w≥0} ‖Φ_{:,S} w − m‖_2^2 {NNLS}
    r ← m − Φ_{:,S} w
end for
return S, w
```

3.4. Nonuniform Compaction

Different attention heads can exhibit different attention patterns (Wu et al., 2025; Xiao et al., 2025; Bick et al., 2025), suggesting that a fixed compaction ratio across all heads is suboptimal. We say that *uniform* compaction uses the same ratio for every KV-head at every layer, whereas *nonuniform* compaction assigns a potentially different ratio to each head and layer.[3]

To motivate a reusable nonuniform compaction schedule for each model, we first show that attention-head sensitivity is largely input-invariant—although absolute loss varies by example, the relative ranking of head importance remains stable. Figure 2 shows the resulting sensitivity curves for Qwen3-4B, averaged over contexts. These rankings also transfer across datasets; the same qualitative pattern appears on LongHealth (Appendix E).

Precomputed head budgets. This stability enables us to precompute, once per model, a nonuniform compaction schedule—a per-head *share* of the total KV budget—that can be reused across contexts and compaction ratios. Concretely, given the per-head sensitivity curves, we solve a discrete resource allocation problem using a standard greedy exchange algorithm (Algorithm 4). Starting from a uniform allocation across heads, we iteratively swap units of KV budget between heads to minimize loss implied by the sensitivity curves. We repeat this process until no swap yields further improvement. This procedure assumes approximate

[3] A naïve nonuniform cache implementation would pad all heads to the length of the longest KV-head, increasing the effective context length and partially negating compute savings. However, attention kernels that support variable-length sequences (e.g., FlashAttention (Dao, 2024)) can avoid this overhead by packing per-head KV segments into a flat, variable-length (varlen) representation. With such packing, nonuniform KV caches can achieve the same memory and compute benefits of a uniform cache of the same total size (Feng et al., 2025; Rehg, 2024).